

Microsoft Careers Internship Monitor

Dossier Tecnico di Architettura & Guida alle Domande da Colloquio

1. L'Elevator Pitch: Come presentare il progetto in 60 secondi

«Ho sviluppato un sistema cloud autonomo a costo zero che risolve un problema pratico: monitorare le offerte di stage per studenti sul portale Microsoft Careers senza dover controllare il sito manualmente ogni giorno. Il bot interroga le API REST interne di Eightfold utilizzate dal portale, filtra rigorosamente per livello di esperienza 'Intern', applica un algoritmo di diffing in memoria con complessità $O(1)$ tramite Set matematici per evitare notifiche duplicate, e recapita notifiche istantanee formattate in HTML su Telegram. L'intero ciclo di vita è automatizzato ogni 30 minuti tramite GitHub Actions con persistenza su Git e segregazione delle credenziali tramite GitHub Secrets. L'architettura è basata sul Provider Pattern, rendendo il sistema universale e pronto a supportare altri provider come Google o Amazon con zero refactoring.»

2. Architettura del Software e Principi Applicati

- **Single Responsibility Principle (SRP):** Ogni modulo ha un solo motivo per cambiare. `models.py` definisce i tipi di dominio, `storage.py` gestisce la persistenza, `client.py` la comunicazione HTTP verso Microsoft, `notifier.py` il formatting e le API Telegram, `config.py` le env vars, e `main.py` orchestra il flusso.
- **Data Structures & Performance (Set vs List):** Per verificare se un ID è già stato notificato, usiamo un `Set` matematico basato su Hash Table con lookup $O(1)$ costante, anziché scorrere una lista $O(N)$. Salviamo poi su file ordinando con `sorted(list)` per generare commit deterministici su Git.
- **API Interne vs Web Scraping:** Non usiamo browser pesanti (Selenium/Playwright) che consumano giga di RAM. Interrogiamo direttamente l'endpoint JSON (`api/pcsx/search`) usato dalla SPA di Microsoft con `httpx`, gestendo cookies di sessione, CSRF token e header mimetici.
- **Idempotenza e Gestione del Cold Start:** Se il bot parte la prima volta su un database vuoto, registra lo stato attuale senza spammare notifiche all'utente e imposta `is_initialized=True`. Le esecuzioni successive notificano solo la differenza di nuovi annunci (diffing idempotente).
- **Sicurezza e Segregazione dei Segreti:** Zero token salvati nel repository. Utilizzo di `.gitignore` in locale e GitHub Actions Secrets in cloud, garantendo che il repository possa essere 100% pubblico e sicuro.

3. Domande e Risposte Tipiche da Colloquio Tecnico

D1: Perché hai deciso di strutturare il codice in moduli invece di fare un singolo script 'main.py'?

«Per rispettare il Principio di Singola Responsabilità (SRP) e garantire la manutenibilità del software. Se un domani Microsoft cambia la struttura del JSON delle sue API, devo modificare esclusivamente il file client.py. Se volessi inviare notifiche su Discord o Slack al posto di Telegram, toccherei solo notifier.py. Un monolite procedurale rende il debugging difficile, mentre moduli isolati da 40-60 righe consentono testing unitario e alta leggibilità.»

D2: Come hai trovato l'endpoint JSON non documentato di Microsoft Careers e come gestisci la sicurezza anti-bot?

«Ispezionando il traffico di rete della SPA (Single Page Application) durante la ricerca sul portale apply.careers.microsoft.com. Abbiamo rilevato che Microsoft utilizza il motore Eightfold AI. L'API di ricerca richiede un handshake iniziale per raccogliere i cookie di sessione (`_vs`, `_vscid`) e l'header 'x-csrf-token'. Il nostro client Python simula una normale sessione di navigazione con `httpx`, utilizzando User-Agent e Referer legittimi. Questo ci consente di ricevere payload JSON strutturati senza l'overhead di calcolo di Selenium.»

D3: Come hai scoperto che selezionando 'Experience level: Intern' sul sito c'erano 0 risultati nonostante posizioni aperte?

«È stato un interessante caso di analisi dati reale: ispezionando il bundle JavaScript 'pcsxPwa.js', abbiamo scoperto che i filtri avanzati vengono inviati con il prefisso 'filter_seniority=Intern'. Analizzando le offerte pubblicate a Milano, abbiamo riscontrato che chi inserisce le posizioni nelle HR di Microsoft aveva scritto 'Intern' nel titolo ma aveva ommesso di spuntare il tag formale 'Seniority' nel database. Abbiamo quindi reso i filtri configurabili da `.env`: l'utente può scegliere tra il filtro rigido formale o la ricerca testuale libera.»

D4: Se un domani volessi monitorare anche le offerte di Google Careers, come estenderesti il progetto?

«L'architettura è predisposta per il Provider Pattern. La classe JobPosting è universale e genera un 'unique_id' composito con prefisso aziendale ('microsoft:123', 'google:456'). Per integrare Google, basta creare una classe GoogleCareersClient che restituisce una lista di JobPosting. Il resto dell'infrastruttura (LocalStorage, TelegramNotifier e l'algoritmo di diffing) non richiede alcuna modifica.»

D5: Perché hai scelto Git come sistema di memorizzazione dello stato su GitHub Actions invece di un database?

«È stata una scelta di pragmatismo ingegneristico: per un alert personale ad alta frequenza (ogni 30 min), Git offre persistenza versionata a costo zero assoluto, senza la necessità di dipendere o configurare database esterni. A ogni novità, il runner esegue un commit automatico di 'seen_jobs.json'. Per un'evoluzione a livello enterprise con migliaia di utenti, il passaggio naturale sarebbe un database serverless come Upstash Redis o Supabase con TTL automatico.»

D6: Come garantisci che il tuo bot non violi i limiti di frequenza (Rate Limiting) di Telegram?

«Telegram impone una soglia di sicurezza di massimo 1 messaggio al secondo verso singole chat private e 30 messaggi/sec globali. Nel notifier abbiamo inserito una pausa preventiva (`time.sleep`) di 1 secondo tra l'invio di ciascuna notifica e implementato il metodo `notify_job` con gestione sicura delle eccezioni e sanitizzazione dell'HTML tramite `html.escape` per prevenire errori di parsing.»

D7: Come si comporta il sistema in caso di prolungata assenza di nuove posizioni (mesi senza aggiornamenti)?

«Nel design abbiamo affrontato il problema del 'fallimento silenzioso': se un bot non notifica nulla per mesi, l'utente non può sapere se non ci sono offerte o se il bot è andato in crash. Abbiamo previsto: 1) Un flag 'is_initialized' che evita falsi Cold Start; 2) La predisposizione per un Heartbeat periodico di vitalità; 3) Un workflow di keepalive che impedisce a GitHub Actions di disattivare il cron dopo 60 giorni di inattività del repo.»

D8: Come gestisci la sicurezza dei segreti su un repository pubblico?

«Segregazione totale tra codice e configurazione, secondo i principi delle Twelve-Factor Apps. In locale i segreti risiedono in '.env', rigorosamente escluso dal versionamento tramite '.gitignore'. In cloud su GitHub Actions, le credenziali vengono iniettate a runtime tramite GitHub Actions Secrets (crittografia asimmetrica a riposo), garantendo che il codice sorgente resti 100% pubblico e sicuro nel mio portfolio.»

D9: Perché hai usato una @dataclass in Python invece di un normale dizionario?

«Le dataclass garantiscono Type Safety, leggibilità e autocompletamento nell'IDE. Con un dizionario generico, un errore di battitura su una chiave come 'annuncio["titlo"]' genera un KeyError solo a runtime in produzione. Con la dataclass, il modello è formalizzato, i tipi sono documentati e possiamo incapsulare logica e proprietà calcolate come '@property def unique_id' direttamente all'interno dell'entità.»

D10: Se dovessi scalare questa applicazione per 10.000 utenti con ricerche personalizzate, come cambieresti l'architettura?

«Passerei da un modello batch sincrono su GitHub Actions a un'architettura a microservizi asincrona orientata agli eventi (Event-Driven): 1) Database relazionale (PostgreSQL/Supabase) per salvare utenti, filtri di ricerca e job visti; 2) Ingestion Worker periodico (AWS Lambda o Cloudflare Workers) che interroga le API e pubblica gli annunci su una coda di messaggi (RabbitMQ o AWS SQS); 3) Dispatcher Workers che consumano i messaggi dalla coda, incrociano i filtri degli utenti e inviano le notifiche con batching; 4) Webhook Telegram bidirezionale per permettere agli utenti di impostare i propri filtri direttamente chattando con il bot.»

4. Glossario dei Concetti Chiave da Menzionare

Concetto	Definizione Rapida & Come si applica al progetto
Idempotenza	Un'operazione è idempotente se eseguirla una o dieci volte produce lo stesso risultato. Il nostro diffing assicura che eseguire il bot 10 volte non generi notifiche duplicate.
Single Responsibility	Ogni classe e file fa una sola cosa bene (es. client fa solo HTTP, notifier solo Telegram, storage solo JSON).
Hash Table O(1)	La struttura interna dei Set in Python: lookup istantaneo per verificare se un job_id è già visto, senza scorrere tutta la lista.
Cold Start	La gestione della primissima esecuzione quando la memoria è vuota: si salvano gli annunci attuali come baseline senza inviare decine di messaggi.
Defensive Parsing	Estrarre dati da API esterne usando valori di default (.get()) e controlli di tipo per evitare crash se il formato della risposta cambia.
Session Cookies & CSRF	Simulazione accurata del browser in httpx per superare i requisiti di sicurezza delle moderne Single Page Applications.
Twelve-Factor App	Metodologia per software moderno: rigorosa separazione tra codice e configurazione (token in .env e secrets).